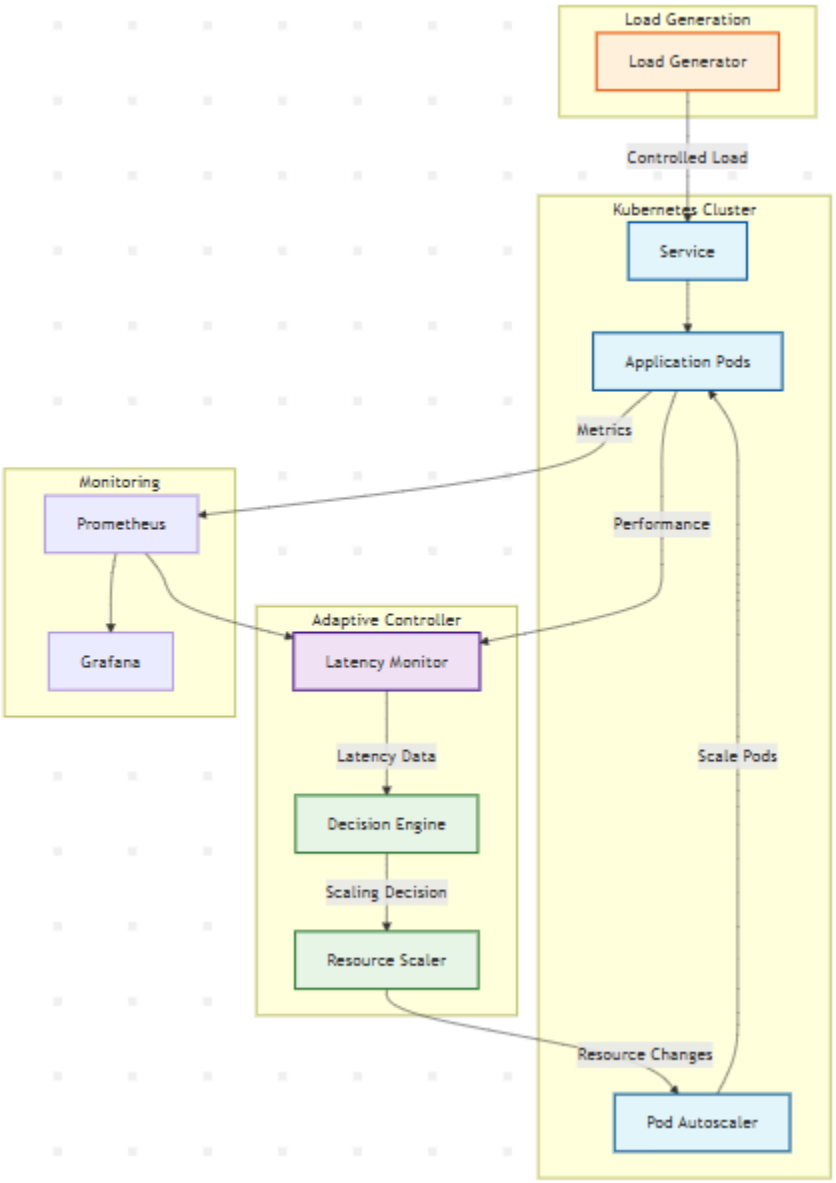
Initial Architecture



High-Level Architecture Overview

1. Initial Deployment -> 2. Stability Phase -> 3. Stress Phase -> 4. Adaptive Vertical Scaling -> 5. Monitoring Loop

Stability Phase – Base Workload Discovery

To automatically discover the most stable request rate (R_0) that our application can handle without overloading the system and while maintaining a low, steady latency. The latency and CPU usage observed at this point form our performance baseline (L_0).

Why This Matters:

- Applying too low a load wastes resources (underutilization).
- Applying too high a load causes latency spikes or failures.
- This phase helps find the "sweet spot" for the application in its current resource envelope (CPU/memory requests).

Algorithm: Auto-Selecting the Stable Load

1. **Start with a Controlled Initial Load**

- Initialize request rate at 1 request per second (RPS).
- Ensure that this is well below what the service can handle.
- Allow the system to stabilize for ~30 seconds to 1 minute.
- Measure and log:
    - p95_latency(t)
    - CPU_Usage(t)
    - Memory_Usage(t)

2. **Ramp-Up Loop (Iterative Load Increase)**

- Loop Logic(Pseudo Code)

```
R = 1  # start RPS
ΔR = 1  # increment in each step
interval = 30 seconds  # time to observe metrics per step

while True:
   apply_load(R)
   wait(interval)
   L = get_p95_latency()
   cpu = get_cpu_usage_percent()

   # Check for breaking conditions
   if (L - previous_L) / ΔR > LATENCY_SENSITIVITY_THRESHOLD:
      break
   if cpu > MAX_CPU_THRESHOLD:
      break

   previous_L = L
   R += ΔR
```

- The division:

- (L - previous_L) / ΔR represents the rate of change of latency with respect to request rate — essentially, the "latency gradient".
- It tells us, "How much does the latency increase per each additional request per second (RPS)?"
- Why we use it:
    - To Detect Performance Saturation: As we increase RPS, the service initially handles it well (small latency changes). But when we approach resource limits (CPU/memory/thread pool), even a small increase in RPS causes a big jump in latency. This formula helps detect that inflection point.
    - Gradient Helps Compare Across Services: By calculating the rate of change rather than the absolute change, we make the condition:
        - Scalable across slow and fast services
        - Independent of absolute latency numbers
        - Easier to compare
    - It's Like a Derivative (Δy/Δx): It's conceptually the first derivative of latency w.r.t. request rate: How fast is latency growing when load increases? If this slope (gradient) becomes too steep, we stop ramping.
- Why Not Just Use Latency Alone: Because raw latency can fluctuate, and a high latency at one point might not be a trend. This gradient checks whether latency is becoming sensitive to load, not just high.
- LATENCY_SENSITIVITY_THRESHOLD
    - We can define the LATENCY_SENSITIVITY_THRESHOLD using our initial latency (L') by expressing it as a percentage of acceptable latency increase per additional RPS. This method is adaptive and scales with the responsiveness of our service.
    - LATENCY_SENSITIVITY_THRESHOLD = α × L', Where: L' = Initial latency (e.g., p95 latency under stable initial load), α = Sensitivity factor (usually between 0.03 and 0.10)

Service Type	Suggested α	Meaning
Real-time Systems	0.03 (3%)	Sensitive to latency increases
Web APIs / SaaS	0.05 (5%)	Moderately sensitive
Async/Background APIs	0.07–0.10	More tolerant

- Why This Works:
    - It adapts to both fast and slow services.
    - Makes our ramp-up logic generalizable to different microservices.
    - Prevents arbitrary hardcoded thresholds that may not fit all services.

- How to calculate initial latency fro the threshold definition:

- To find the base latency (L') initially, you need to follow a controlled, data-driven process right after deployment and before any optimization or scaling begins.
- How to Find Initial Latency (L')
  - Start the Service with Minimal Load: Deploy the application with default CPU/memory (e.g., 250m CPU, 256Mi RAM). Set the initial request rate to something low, like 1 RPS.
  - Warm Up the Service: Let the service run for ~2–5 minutes(This allows JIT warmup (for JVM-based apps), caching, internal thread pool startup, etc.)
  - Start Monitoring p95 Latency: Using Prometheus(histogram_quantile(0.95,rate(http_request_duration_seconds_bucket[1m])))
  - Ensure Latency is Stable: Check that latency doesn't drift upward over time. Use linear regression on the last 1–5 minutes of latency values to ensure slope ≈ 0.

    ```
    from sklearn.linear_model import LinearRegression
    import numpy as np

    X = np.arange(len(latencies)).reshape(-1, 1)
    y = np.array(latencies)
    slope = LinearRegression().fit(X, y).coef_[0]

    if abs(slope) < 0.5:  # ~0.5ms/sec
        latency_is_stable = True
    ```

  - Set Initial Latency (L'): Once stable(L' = average(p95_latency over stable window). If our latency has noise (e.g., jumps between 110 and 150 ms), use p95 of a moving window instead of an average.
- Practical Parameters

Parameter	Suggested Value	Description
Start RPS	1	Initial low load
RPS Increment ΔR	+1 or +5	Finer steps give more precise control
Wait Interval	0 sec – 1 min	Wait before measuring impact
Latency Threshold (ΔL/ΔR)	α × L'	Defines system instability slope
Max CPU Threshold	75–80%	Avoid saturation

- Metrics to Monitor During Each Step
    - p95_latency – Use high-percentile (not average) latency to capture user-facing performance degradation.
    - ΔL/ΔR (Latency Gradient) – Detects sharp increases in latency as load increases.
    - CPU Usage (%) – Helps avoid resource starvation or throttling.
    - *(Optional)* GC Pause Time, Thread Queue Length – JVM-based services especially.
- Breaking Conditions — When to Stop Ramp-Up
    - Stop increasing load when:
        - Latency becomes sensitive to load
          $\Delta R / \Delta L = (L_{current} - L_{previous}) / (R_{current} - R_{previous}) > \text{Threshold}$
          This means system is getting saturated or queuing is beginning to dominate response time.
        - CPU usage exceeds safe threshold
          CPUUsage > Max threshold
          This indicates approaching saturation or throttling (especially in Kubernetes).

3. **Backtrack to Last Stable Point**
   Once a breaking condition is hit:
    - Rollback to previous RPS (R_0)
    - Record:
        - R_0 = max sustainable request rate
        - L_0 = corresponding p95 latency
        - cpu_0 = CPU usage at R_0
    - This becomes your performance baseline
4. **Smart Stability Check**
   To avoid false detection due to jitter or noisy metrics, apply statistical smoothing. Use Linear Regression:
    - On a 5-minute sliding window of latency (p95_latency(t))
    - Fit latency vs time using least squares
    - If slope ≈ 0, latency is stable

```
from sklearn.linear_model import LinearRegression
window = last_5_minutes_latency_values
X = timestamps.reshape(-1, 1)
y = latencies

model = LinearRegression().fit(X, y)
slope = model.coef_[0]

if abs(slope) < 0.5:  # milliseconds per second
   latency_is_stable = True
```

Why This Works:

- Balances real performance (latency) with infrastructure pressure (CPU).
- Avoids over/under-provisioning before scaling logic begins.
- Produces ground truth for later comparison during stress or optimization phases.

After This Phase:

We now have:

- A stable workload (R_0)
- A target latency (L_0)
- System pressure metrics (CPU_0, $Memory_0$)

Stress Phase – Performance Deviation Analysis

After identifying the stable base request rate (R_0) and base latency (L_0) in the previous phase, the goal of this phase is to:

- Apply a higher workload ($R_1 > R_0$)
- Measure how the system responds in terms of latency and resource pressure
- Trigger vertical scaling if latency worsens
- Explore optimization if performance improves

Input from Previous Step:

- R_0 -> Base request rate (max sustainable rate without degradation)
- L_0 -> Corresponding base p95 latency
- Optional: cpu_0, mem_0 -> CPU and memory usage at baseline

Increase the Request Rate

Apply a higher request rate than the base: $R_1 = R_0 + \Delta R$

We can choose ΔR as:

- +20% if you're testing moderate load increase
- +50–100% for aggressive surge testing

Let System Stabilize

Once R_1 is applied:

- Let the system run for a fixed time window (e.g., 1–3 minutes)
- This allows caching, CPU ramp-up, GC, and any retries to stabilize

Measure Key Metrics

Record:

- L_1 = p95 latency under R_1
- cpu_1 = CPU usage %
- mem_1 = Memory usage %
- Optional: GC time, queue depth, error rates

Compare Against Baseline

Case 1: Performance Degraded

if $L_1 > L_0$:

performance_degraded = True

- Latency increased — system is struggling under R_1.
- This indicates:
    - CPU/memory bottlenecks
    - Saturated thread pools
    - GC pressure
    - DB/backend constraints

Case 2: Performance Improved or Same

if $L_1 <= L_0$:

performance_stable_or_better = True

- Latency is stable or even lower than L_0 under higher load.
- This may mean:
    - The app is under-provisioned at R_0
    - JVM warmed up
    - Batching/pipelining became efficient
    - Thread pools now fully utilized

Why Latency Can Drop under Higher Load

- Low concurrency can keep workers idle.
- Higher load can improve thread pool utilization, cache hits, or batching.
- For ML inference, batching larger requests can reduce per-request overhead.
- Lower latency at higher RPS ≠ bug — may mean system is underloaded.

What This Step Achieves

- Pushes system toward peak performance
- Lets us dynamically decide whether to scale, optimize, or update baseline
- Prepares the system for realistic, fluctuating workloads

Adaptive Vertical Scaling (with Bayesian Optimization)

Use Bayesian Optimization to smartly adjust CPU and memory requests for a Kubernetes pod to:

- Restore degraded performance (if $L_1 > L_0$)
- Improve performance further (if $L_1 < L_0$)
- Avoid overprovisioning or underutilization
- Minimize resource usage while keeping latency near or below L_0

Why Bayesian Optimization?

Unlike greedy step-wise increments, Bayesian Optimization:

- Explores intelligently based on past performance
- Balances exploration (try new CPU/mem combos) and exploitation (focus near best)
- Avoids wasteful trials and quickly converges to near-optimal allocations

Inputs:

From previous steps:

- R_0 = Base request rate
- R_1 = Applied request rate ($R_1 > R_0$)
- L_0 = Base latency
- L_1 = Current latency under stress

System Design

- Objective function:
  Minimize latency + resource usage penalty
- Parameters to tune:
  - cpu_request (e.g., from 200m to 1500m)
  - mem_request (e.g., from 256Mi to 2Gi)

Step-by-Step Execution

1. **Define the Objective Function**

   def objective(trial):

   # Suggest CPU and memory request values

   current_cpu = get_current_cpu_request() # e.g., 0.5 cores

   current_mem = get_current_memory_request() # e.g., 1.0 Gi

```
cpu_min = current_cpu * 0.5
mem_min = current_mem * 0.5

cpu_m = trial.suggest_float("cpu", cpu_min, cpu_max)
mem_g = trial.suggest_float("memory", mem_min, mem_max)

# Apply the resource patch to the deployment
apply_patch_deployment(cpu=cpu_m, mem=mem_g)

# Wait for pod to stabilize
wait_for_stabilization()

# Measure current latency and resource usage
latency = measure_p95_latency()
cpu_util = measure_cpu_usage_pct()
mem_util = measure_memory_usage_pct()

# Normalize if usage is in 0–100%
if cpu_util > 1: cpu_util /= 100
 if mem_util > 1: mem_util /= 100

# Compute performance gap
latency_penalty = max(latency - initial_latency, 0)
```

```
    # CPU utilization penalty (dual-sided)
    # Optimal CPU usage range: 60% to 85%
    if cpu_util < 0.6:
        cpu_penalty = (0.6 - cpu_util) * 10
    elif cpu_util > 0.85:
        cpu_penalty = (cpu_util - 0.85) * 15
    else:
        cpu_penalty = 0

    # Memory utilization penalty (dual-sided)
    # Optimal Mem usage range: 60% to 85%
    if mem_util < 0.6:
        mem_penalty = (0.6 - mem_util) * 5
    elif mem_util > 0.85:
        mem_penalty = (mem_util - 0.85) * 10
    else:
        mem_penalty = 0

    # Final score: minimize this
    score = latency_penalty + cpu_penalty + mem_penalty
    return score
```

2. Run the Optimizer (e.g., with Optuna)

   Use Optuna's study.stop() method from inside the objective function or a custom callback when:

- Latency reaches below a target
- Best score hasn't improved in N trials
- We detect instability

```
import optuna

study = optuna.create_study(direction="minimize")


def early_stopping_callback(study, trial):
    if study.best_value is not None and study.best_value < 0.01:
        print("Early stopping: target performance achieved.")
        study.stop()


study.optimize(objective, n_trials=1000, callbacks=[early_stopping_callback])
```

This will:

- Try various CPU/Memory settings
- Observe their impact on latency and utilization
- Find the most balanced configuration

3. Analyze and Apply Best Config

```
best = study.best_params

cpu_best = best["cpu"]

mem_best = best["memory"]

apply_patch_deployment(cpu=cpu_best, mem=mem_best)
```

Condition-Based Control Logic

Case A: If $L_1 > L_0$ (Performance Degraded)

- Use the above optimizer to scale up CPU/memory until latency improves.
- Use latency_penalty to heavily prioritize regaining performance.

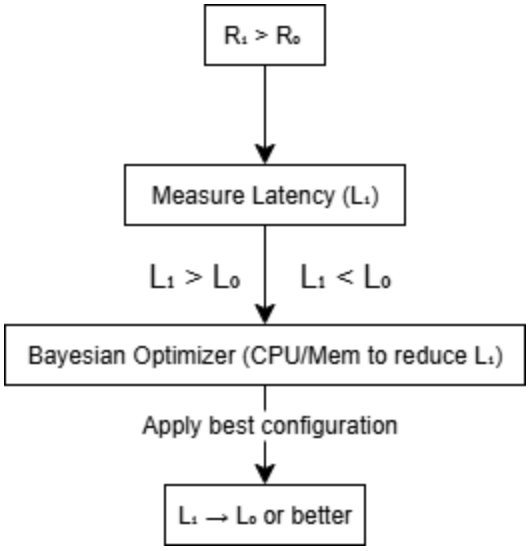
Case B: If $L_1 < L_0$ (Latency Improved)

- Use the optimizer to minimize latency further while reducing resources.
- Let the optimizer explore smaller CPU/mem values.
- Penalize waste (low utilization) to push resource efficiency.

Benefits of This Approach

Feature	Greedy	Bayesian Optimization
Finds global minimum	No	Yes
Avoids overprovisioning & underprovisioning	May be	Yess
Works under noisy data	No	Yes
Fast convergence	No	Yes (with few trials)
Easy to automate	Yes	Yes

Flow



Optional: Monitoring & Learning Loop (Reinforcement-Inspired Feedback System)

Goal

Create a self-adaptive system that continuously:

- Observes runtime performance
- Learns how different CPU/memory settings impact latency
- Builds a knowledge base of best configurations for various workloads
- Uses that knowledge to preemptively optimize for future workload changes

Key Concepts

Component	Description
Latency gap (ΔL)	The difference between current latency and base latency (L_1 - L_0)
Resource headroom	The amount of unused CPU or memory (i.e., underutilized allocations)
Workload signature	Defined by request rate (RPS), time of day, or workload category
Knowledge base (KB)	A map of workload → best resource config (CPU/mem)
Feedback frequency	How often the loop executes (e.g., every 5 minutes)

Step-by-Step Monitoring & Learning Loop

1. Every N Minutes (e.g., 5 Min): Collect Metrics

   Use Prometheus
   Collected metrics:

   ```
   latency = measure_p95_latency()
   cpu_usage = measure_cpu_usage_pct()  # actual usage %
   mem_usage = measure_memory_usage_pct()
   cpu_request = get_requested_cpu()  # allocated request (cores)
   mem_request = get_requested_memory()  # in Gi
   rps = measure_request_rate()
   ```

2. Compute State Features

   Define system performance state:

   ```
   latency_gap = latency - base_latency        # ΔL
   cpu_headroom = 1 - cpu_usage                # 0.3 means 30% unused
   mem_headroom = 1 - mem_usage
   utilization_ratio = (cpu_usage + mem_usage) / 2
   ```

3. Evaluate Current Efficiency Score

   ```
   cpu_price_ratio = get_current_cpu_cost_per_core()
   mem_price_ratio = get_current_memory_cost_per_GB()

   total_price = cpu_price_ratio + mem_price_ratio

   cpu_penalty_weight = cpu_price_ratio / total_price
   mem_penalty_weight = mem_price_ratio / total_price
   latency_weight = 1.0  # always high

   score = (
      max(latency_gap, 0) * latency_weight
      + cpu_headroom * cpu_penalty_weight
      + mem_headroom * mem_penalty_weight
   )
   ```

   Lower score = better system state

4. Define Workload Signature

   Create a unique key to represent the workload context:

   ```
   workload_key = {
   ```

```
    "rps": round(rps, 1),
    "time_window": get_current_time_window(),  # e.g., morning, peak, off-peak
    "service": service_name
}
```

5. Update Knowledge Base (KB)

   Knowledge base stores mappings like:

```
{
  "rps": 15.0,
  "time_window": "peak",
  "service": "service-1"
} => {
  "cpu": 0.6,
  "memory": 1.25,
  "score": 5.2
}
```

   Update Logic:

```
if workload_key not in knowledge_base:
    knowledge_base[workload_key] = current_config
else:
    if score < knowledge_base[workload_key]["score"]:
        knowledge_base[workload_key] = current_config  # improved config
```

6. Action: Recommend or Apply Better Config

   On next loop or during autoscaling:

```
if workload_key in knowledge_base:
    best_config = knowledge_base[workload_key]
    if current_config != best_config:
        apply_patch_deployment(cpu=best_config["cpu"], mem=best_config["memory"])
```

   This enables the system to self-optimize proactively, not reactively.

Optional: Model the Latency-Resource Surface

As data accumulates, fit a predictive model:

latency = f(cpu, memory, rps) + ε

Use:

- Regression Trees
- Gaussian Process
- Neural Network
- Bayesian Surface Estimation

This enables:

- Predicting what CPU/mem combo will keep latency near L_0
- Simulating impact before deploying change

System Architecture Flow

[Metrics Collection] -> [Compute Score & Headroom] -> [Identify Workload Signature (RPS, Time)] -> [Check & Update Knowledge Base] -> [Recommend / Apply Better Config] -> [Wait N Minutes & Repeat]

Feature	Benefit
Continuous learning	Adapts to workload shifts
No manual tuning	Builds optimal resource map over time
Avoids overprovisioning	Detects and corrects waste automatically
Preemptive autoscaling	Applies best config before degradation begins

Optional: How to Connect Bayesian Surface Estimation to Our System

Step 1: Collect Training Data

From each Monitoring Loop iteration, store:

X = [cpu_request, mem_request, rps]
y = [observed_latency]

Build a dataset

Step 2: Train a Bayesian Regression Model

Gaussian Process Regression (GPR):

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, WhiteKernel
import numpy as np

# X: [cpu, memory, rps]
# y: latency
X = np.array(training_data_inputs)  # shape: (n_samples, 3)
y = np.array(training_data_outputs)  # shape: (n_samples,)

kernel = RBF(length_scale=1.0) + WhiteKernel(noise_level=1)
model = GaussianProcessRegressor(kernel=kernel)
model.fit(X, y)
```

This model now estimates latency = f(cpu, memory, rps) ± uncertainty

Step 3: Predict Latency for Candidate Configurations

```
# Predict latency at 0.6 cores, 1.2 Gi, and 14 RPS
latency_pred, std = model.predict([[0.6, 1.2, 14]], return_std=True)

print(f"Expected latency: {latency_pred[0]:.2f} ms ± {std[0]:.2f}")
```

Step 4: Use Predictions in the Feedback Loop

Before deciding to apply a config:

1. Generate several candidate configurations near the current point
2. Predict latency using GPR
3. Choose the one with:
    - lowest latency
    - minimal CPU/memory
    - within a confidence range (e.g., std < 20 ms)

```
candidates = [
    [0.5, 1.0, rps],
    [0.6, 1.0, rps],
    [0.7, 1.2, rps],
    ...
]

best_score = float("inf")
best_config = None

for config in candidates:
    pred, std = model.predict([config], return_std=True)
    if std[0] < 15:  # avoid high uncertainty
        score = pred[0] + 5 * config[0] + 3 * config[1]  # latency + weighted CPU/mem cost
        if score < best_score:
            best_score = score
            best_config = config

# Apply best_config: [cpu, memory, rps]
```

apply_patch_deployment(cpu=best_config[0], mem=best_config[1])

Step 5: Continually Retrain the Model

Every N iterations (e.g., every 10 samples), retrain:

```
if len(training_data_inputs) % 10 == 0:
    model.fit(X, y)
```

This keeps the surface estimation up-to-date as load patterns change.

Benefit	Description
Predicts before deploying	Reduces unnecessary patching and rolling restarts
Learns a generalized latency surface	Not just for one RPS — works across any workload
Handles uncertainty	Uses ± std to skip bad recommendations
Integrates seamlessly	Fits directly into your Step 5 feedback loop

Tools We Can Use

Tool	Purpose
scikit-learn	GPR, Bayesian Ridge
GPy, GPyTorch	More scalable GPs
Optuna	Bayesian optimizer with study.sampler
BayesianOptimization package	High-level interface to fit surfaces

Architecture Update (with Bayesian Layer)

[Metrics Collector] -> [Training Data (X, y) Store] -> [Bayesian Surface Estimator (GPR)] -> [Predict latency for candidate configs] -> [Choose optimal (CPU, mem) combo] -> [Patch Deployment + Feedback]